Mining Software Repositories

Chapter 4: Developer Social Networks

Prof. Dr. Steffen Herbold

Contents

- Social networks and graphs
- Developer networks
- Network metrics

Social network

- Definition according to Meriam-Webster:
  1) a network of individuals (such as friends, acquaintances, and coworkers) connected by interpersonal relationships
  2) an online service or site through which people create and maintain interpersonal relationships

- Research is mostly interested in the first part of the definition
  - Who works together?
  - Which teams are there?
  - How are teams forming/evolving?
  - What is the role of a developer?
  - …

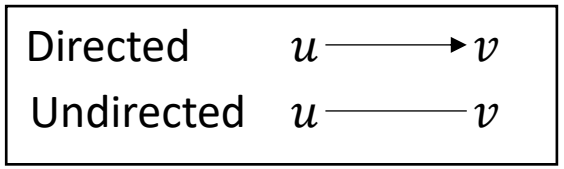
- There are also social network aspects to software engineering tooling itself
  - For example, GitHub has many features we know from social networks, e.g., following people/projects, creating groups, etc.
  - Sometimes studied as well, but more often we just exploit these features as data source to understand relationships

Graphs

- Formally, social networks are defined as graphs
  - While we assume that everybody here knows how to work with graphs, we review the terminology here

- A graph $G = (E, V)$ is a tuple where V defines the vertices/nodes of a graph and $E \subseteq V \times V$ the edges.

- Graphs can be directed or undirected
  - In a directed graph, we can describe an edge as an ordered tuple, i.e., $e = (u, v)$
  - In an undirected graph, we can describe an edge as a set, i.e., $e = \{u, v\}$



  - For simplicity, we use the notation for directed graphs in the following. Any undirected graph can be considered as a directed graph by just adding two directed edges
    - $\{u, v\} \rightarrow (u, v)$ and (v, u)

Properties of graphs

- A *path* of length m in a (directed) graph may be defined in two ways:
  - A sequences of edges $\left(e_1 = \left(v_{1,1},, v_{1,2}\right),\ \ldots, e_m = \left(v_{m,1}, \ldots, v_{m,2}\right)\right)$ such that $v_{i,2} = v_{i+1,1}$ for all $i = 1, \ldots, m-1$
  - A sequence of nodes $(v_1, \ldots, v_m)$ such that $(v_i, v_{i+1}) \in E$ for all $i = 1, \ldots, m-1$
  - The length of a path $d(u, v)$ between two nodes $u, v \in V$ is defined as the number of edges on that path
  - Definitions for undirected graphs are analogous
- Two nodes are *connected*, if a path between them exists

- The *degree of a node* defines the number of edges to which the note is connected
  - In an undirected graph, a node has a degree $deg(v) = \#\{e \in E : v \in e\}$
  - In a directed graph, a node has
    - an out-degree $deg_{out}(v) = \#\{(v_1, v_2) \in E : v = v_1\}$ and
    - in-degree $deg_{in}(v) = \#\{(v_1, v_2) \in E : v = v_2\}$

Properties of graphs

- A graph is called *connected* if there exists a path for all pairs of node $u, v \in V$
  - Otherwise, the graph is called *disconnected*

- A *connected component* of a graph is maximal connected subgraph
  - In this sense, maximal means that we cannot add another node to the subgraph without the subgraph becoming disconnected

- A graph is called *complete* if there exists an edge between all nodes
  - For all $u, v \in V$ with $u \neq v$ exists $(u, v) \in E$

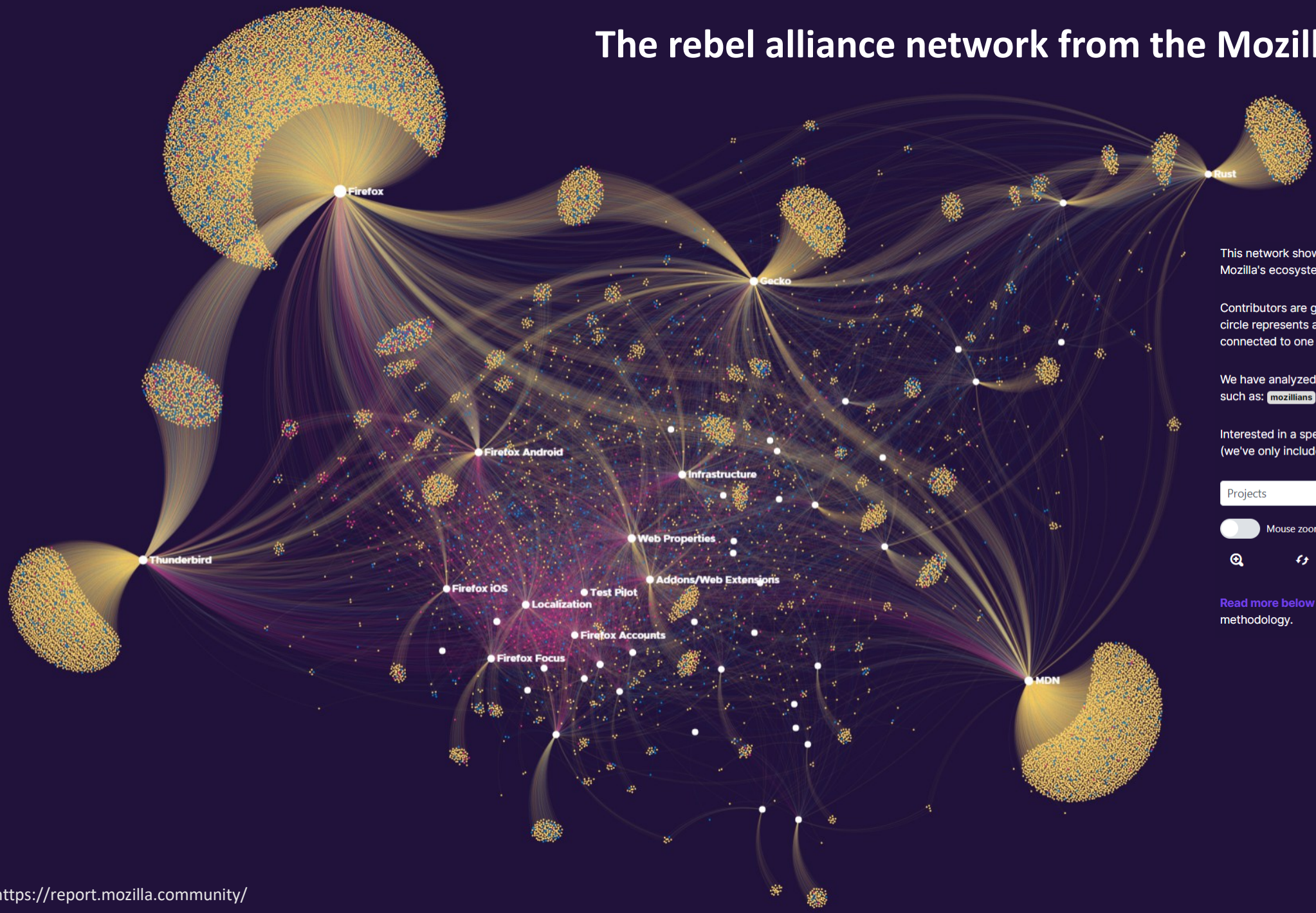
- A subgraph $G' = (V', E')$ is induced by a subset of nodes
  - $V' \subset V$
  - It contains the edges $E' = \{(u, v) \in E : u, v \in V\}$

Communities and partitions

- Graphs can have communities
  - Communities are clusters of nodes which have many connections between them
    - They do not necessarily be fully connected
    - A community can have connections in form of edges to other communities
      - Just not too many, because then the communities would be merge into one large community
  - There is no fixed definition of community in terms of how many internal connections are required and how many external connections are allowed
  - There are different algorithms to detect communities, e.g., by looking at (weakly) connected components or local clustering coefficients
    - We do not discuss this in detail in this lecture
- Formally, a community is just a subgraph and a community detection algorithm determines subgraphs $P = \{p_1, \dots, p_k\}$ of a graph $G = (V, E)$ such that
  - $p_i = (V_i, E_i)$ with $V_i \subseteq V$ and E_i the edges of G induced by the nodes V_i
  - $V_i \cap V_j = \emptyset$ for all $i, j \in \{1, \dots, k\}, i \neq j$
  - $\bigcup_{i=1}^{k} V_i = V$
- We then also say that P is a partitioning of G

Developer social networks

The rebel alliance network from the Mozilla Foundation



This network shows where **22,821** contributors participated across Mozilla's ecosystem between 2017-2019.

Contributors are grouped into segments based on their activity. Each circle represents a casual, regular or core contributor, which is connected to one or more projects. Click on one to see more.

We have analyzed contributions from some affiliate groups in Mozilla, such as: mozillians, tech speakers, mission driven mozillians and OSSN.

Interested in a specific project? Have a look at one of these 49 projects (we've only included detailed information on 10 of them).

Projects

Mouse zoom

Read more below for a detailed description of this work and methodology.

https://report.mozilla.community/

Social, technical, socio-technical

- There is not just one way to model a social network in software development, but many of them
- We can model relationships between people
  - Purely social network
- We can model relationships between technical artifacts
  - This does not (necessarily) mean dependencies, but also other aspects like co-change
  - Technical network
- We can model both together, i.e., how people interact considering a relation to artifacts
  - Social-technical network
- Note that technical networks are also often refered to as socio-technical because often some human interaction is used to derive the relationships

Defining networks for research

- The definition of any social network relies on two key aspects:

1) What are the nodes?

In other words: relationships between which actors/artifacts are we interested in?

2) What are the edges?

In other words: how do we define that there is a relationship?

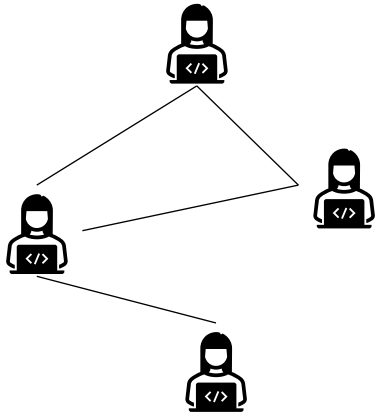
How both is done is similar to what we discussed for direct/indirect measurement:
Sometimes, we can model the actors/artifacts/relationships directly, sometimes we have to do this indirectly

- For example, we often cannot know who talked to whom exactly, because we cannot monitor everything (e.g., private messages, email). But we can indirectly model this based on observable data

Developers as nodes (social network)

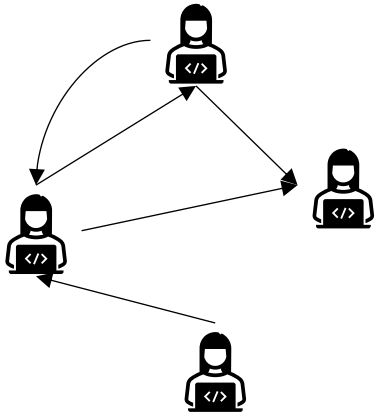
- Often, the developers are the nodes within a developer social network
  - Otherwise, the name would also be really strange
- When developers are nodes, edges are usually related to the communication between these developers
- The concept of communication is interpreted broadly
  - We have the obvious direct communication, e.g., via emails on a mailing list or through discussions within issues
  - We also have indirect communication via software artifacts
- Both can be modelled in a directed and undirected manner

Examples for developers as nodes



Nodes: actors in an issue tracker
Edge: commented in same issue

Not directed, i.e., does not consider order of comments



Nodes: actors in an issue tracker
Edge: commented in same issue

Directed to indicate order of comments. Outgoing edge means commented before.

→ This means the direction of an edge represent knowledge and information flow

The directed network allows to identify whether communication was uni-directional or bi-directional.

Artifacts as nodes (technical network)

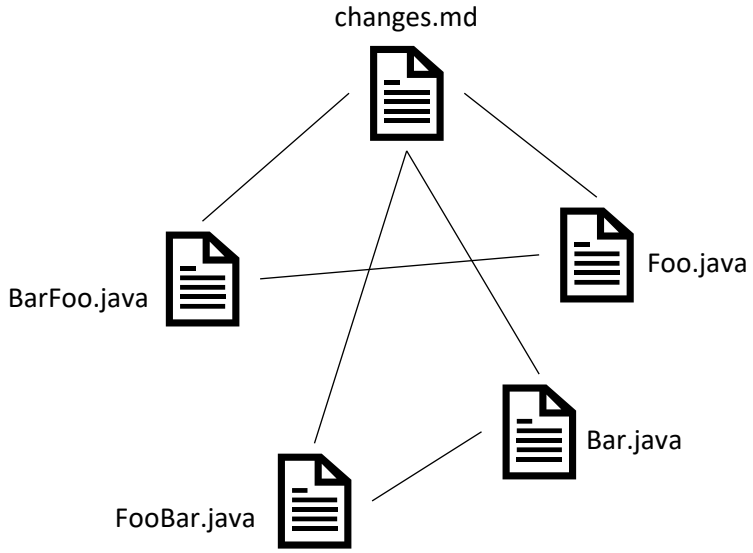
- When we consider artifacts as nodes, we usually consider files

- Networks about files try to model relationships between files
  - Dependency graphs can be viewed as a technical network

- MSR also often looks beyond that to process aspects

- For example, we might want to know what was changed together as some sort of social relationship between files
  - Two files might not technically depend on each other, but are often changed together
  - For example, two classes that implement the same interface might be changed together if the assumption on the behavior of the interface change
  - However, there is no direct technical dependency between these classes

Example for artifacts as nodes

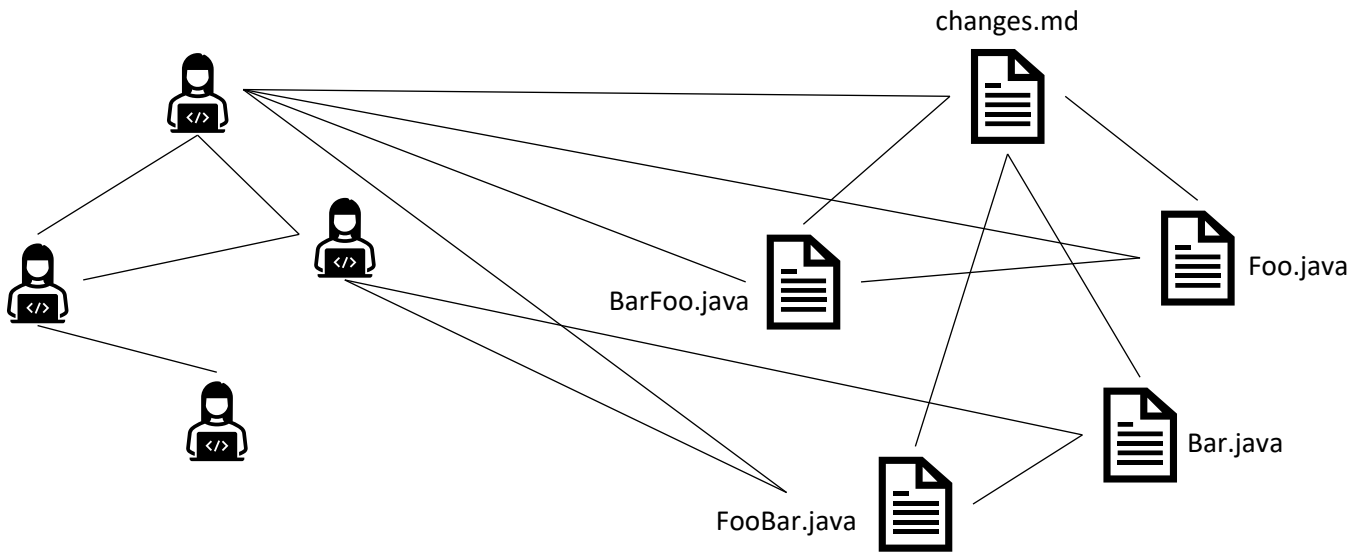


Co-change graph: edges denote files where change together in a commit

Artifacts and developers as nodes (socio-technical network)

- We can also model the relationship between artifacts and developers
- Through this, we can understand the relationship between social aspects (i.e., relationships between developers) and technical artifacts (e.g., code)
- For example, to understand which developers work on the same files we can model them together in the same graph
- We can even combine connection between developers and artifacts, with connections between developers and connections between artifacts to model complex relationships

Example for developers and artifacts as nodes



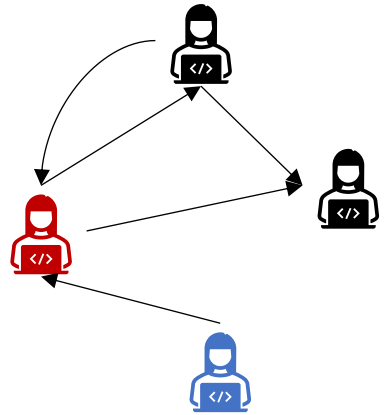
Edges between devs: commented in the same issue
Edges between files: changed together in a commit
Edges between devs and files: dev changed file

Types of nodes

- Nodes can have different types, e.g.,
  - Regarding involvement, e.g., core developers and peripheral developers
  - Regarding activities, e.g., committers and non-committing contributors (e.g., issue reporter)

- If the source of the type is outside of the social network, the types are defined by a function over the nodes
  - $type^V : V \rightarrow types$ where $types$ is a pre-defined set of types
  - Such externally defined types can, e.g., be roles in a project or organization

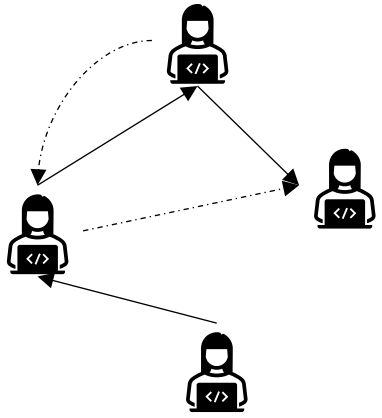
- If the source of the type is the position of the developer in the network, the types are defined by a function over the graph
  - $type^V_{social} : V \times E \rightarrow types$
  - Such types are typically related to how “important” a node is in a network, e.g., high degree
    - More in this when we discuss metrics



Types of edges

- We can also have edges of different kinds

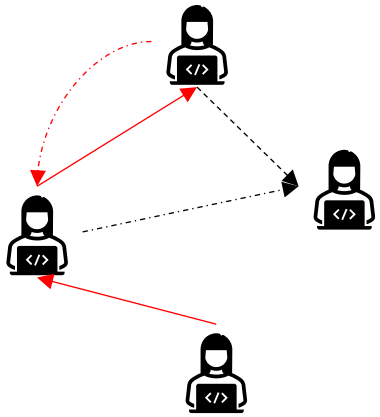
- This is the case when we want to model different kinds of relationships
  - This is usually used to preserve complex information
  - E.g., when we want to create a complex model of communication behavior, we might want to preserve if communication happened via email, issues, pull requests, code, …



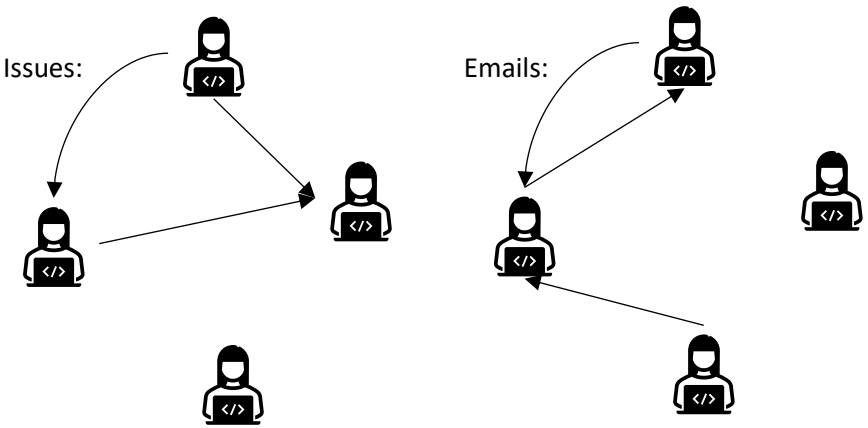
Modeling with multiple edge types

- There are two general cases when dealing with edge types
  - There can only be a single edge between two nodes and each edge can only have a single type
  - Edges between nodes can have multiple types and/or we can have multiple edges between nodes
- Both cases can be modeled by a function over the edges
  - $type^E : E \rightarrow types$
  - In case edges can have multiple types, this function is not injective
- When we have multiple types, it is sometimes easier to model this within different graphs without types
- Example: We have two types of edges
  - Red lines represent email communication
  - Dashed lines represent communication in an issue tracker

Variant 1: single graph with multiple edge types



Variant 2: two graphs with out types

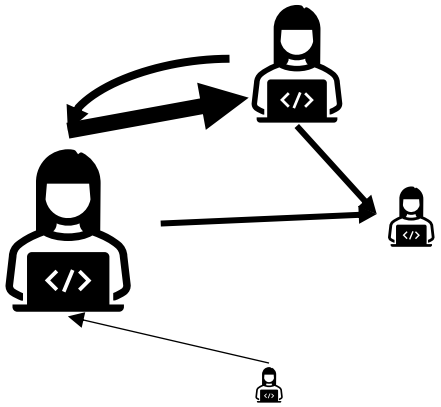


Weights

- Nodes and edges in social graphs can be weighted
  - For nodes, e.g., to include the overall activity
  - For edges, e.g., to include the strength of a relationship

- Implemented trough weighting functions
  - $weight^V: V \rightarrow \mathbb{R}$
  - $weight^e: E \rightarrow \mathbb{R}$

- Usually visualized through size of nodes and thickness of edges



- We can also have multiple weights per edge, i.e., one weight per edge type
  - In this case, it is usually the better solution to work with multiple graphs that do not have edge types
  - Otherwise, we would need to encode this in a joint weight/type function

Network metrics

Measuring graphs

- For analysis, we often want to measure something

- Consequently, we need measures for social networks
- Since social networks are graphs, we need measures for graphs

- Typically, we are interested in one of two aspects:
  - Some property of a graph to understand an aspect of the complete social network

  - Some property of a node to understand properties of the artifacts within the social network

Basic properties

- The basic properties of graphs are important to report and often give valuable insights
- Measures for the size
  - Number of nodes $|V|$
  - Number of edges $|E|$
- Basic structural information
  - Number of components
- These help us to understand the size, the overall number of relations, and if we actually have a single network or disconnected subnetworks

Centrality

- We are often interested in the “importance” of a node for a graph
- One way to define interesting is that a node is somehow central in a graph
- There are different ways to measure centrality. We consider three of them:
  - Degree centrality
  - Closeness centrality
  - Betweenness centrality
- There are also others that, e.g., based on the eigenvectors of the adjacency matrix

Degree centrality

- A simple approach to define how central is node is by just looking at how many connections to others it has
  - This is exactly what the node degree measures!
- We can thus define:
  - $C_{degree}(v) = d(v)$
- For a directed graph, we have two such centrality measures, i.e., one for the in-degree and one for the out-degree
- We can also generalize this concept to the whole graph
  - $C_{degree}(G) = \frac{\sum_{v \in V} |C_{degree}(v^{max}) - C_{degree}(v)|}{|V|^2 - 3|V| + 2}$
    - v^{max} is the node with the highest degree centrality, i.e., $v^{max} = \text{argmax}_{v \in V} C_{degree}(v)$
    - The denominator is the largest possible value that the nominator could achieve, i.e., it normalizes to the interval $[0,1]$
    - This is achieved by a star graph, i.e., a graph with one central node, to which all other are connected. Such a node has degree $|V| - 1$, all other would have degree 1 (i.e., their edge to the central node).

Closeness centrality

- Another way to look at centrality is how long the distance to other nodes is
- The idea is that a location is central, if the length of the paths to other nodes is short

- We can formalize this as follows:
  - $C_{closeness}(v) = \frac{|V|-1}{\sum_{u \in V} d(u,v)}$
  - In this case, $d(u, v)$ is the length shortest path between u and v

Betweenness centrality

- Another perspective for centrality is that something is central if you visit it when traveling from one location to another

- For a graph, this means that a node is on many shortest paths between nodes

- We can formalize this as follows
  - $C_{between}(v) = \sum_{s,t \in V : s,t \neq v} \frac{\sigma_{st}(v)}{\sigma_{st}}$
  - σ_{st} is the number of shortest path between a start s and a target t
  - $\sigma_{st}(v)$ is the number of shortest path between s and t, such that v is in the path

Clustering Coefficient

- Another interesting aspect of graphs is how closely connected neighborhoods are
  - Socially, a closely connected neighborhood means that the social contacts are also in contact with each other
- We define the neighborhood N_v of node v as all nodes that are connected with an edge to v
  - $N_v = \{u \in V : (u, v) \in E \text{ or } (v, u) \in E\}$
- This can be formalized by the clustering coefficient
  - The clustering is measured in relation to complete subgraphs
  - A complete subgraph is called a *clique*
- The clustering coefficient counts the edges in the neighborhood of v in relation to the number of edges a clique would have
  - $C_{clust}(v) = \frac{|\{(u,u') \in E : u,u' \in N_v, u \neq u'\}|}{|N_v| \cdot (|N_v| - 1)}$
  - The nominator are the number of edges in the neighborhood without self-loops
  - The denominator computes how many edges a complete subgraph (=clique) would have
    - Note that this is the definition for a directed graph. With an undirected graph the denominator would instead be $\frac{|N_v|(|N_v|-1)}{2}$
- We can also compute this as average for the complete graph
  - $\bar{C}_{clust}(G) = \frac{1}{|V|} \sum_{v \in V} C_{clust}(v)$

Questions?

